
Ergebnisse Übung 2

GPU-Benchmarks: SIMT-Ausführungsmodell und Speicherbandbreite

Jan-Niclas Loosen

Universität Trier

Benchmark Suite

Python-Paket `gpubench` mit CLI

Im Grunde ein `pyopencl`, `numpy` und `matplotlib` Wrapper

Drei OpenCL-Kernel: `compute_uniform.cl` & `compute_divergent.cl` (Rechenintensiv für Nr. 1) sowie `memory.cl` (Streaming für Nr. 2)

- NumPy-CPU-Referenz für dieselben Workloads
- Zeitmessung über OpenCL-Profiling-Events, Median aus sieben Läufen

Was kostet Warp-Divergenz?

Rechenintensiver Kernel, feste Zahl von FMA-Schritten (*Fused Multiply Add*)

- Schritt 1: Ab welcher Problemgröße ist die GPU ausgelastet?
- Schritt 2: Was kostet es dann, wenn Lanes eines Warps verschiedene Pfade nehmen?

Messgröße: Durchsatz in GFLOP/s (*Floating Point Operations per Second*)

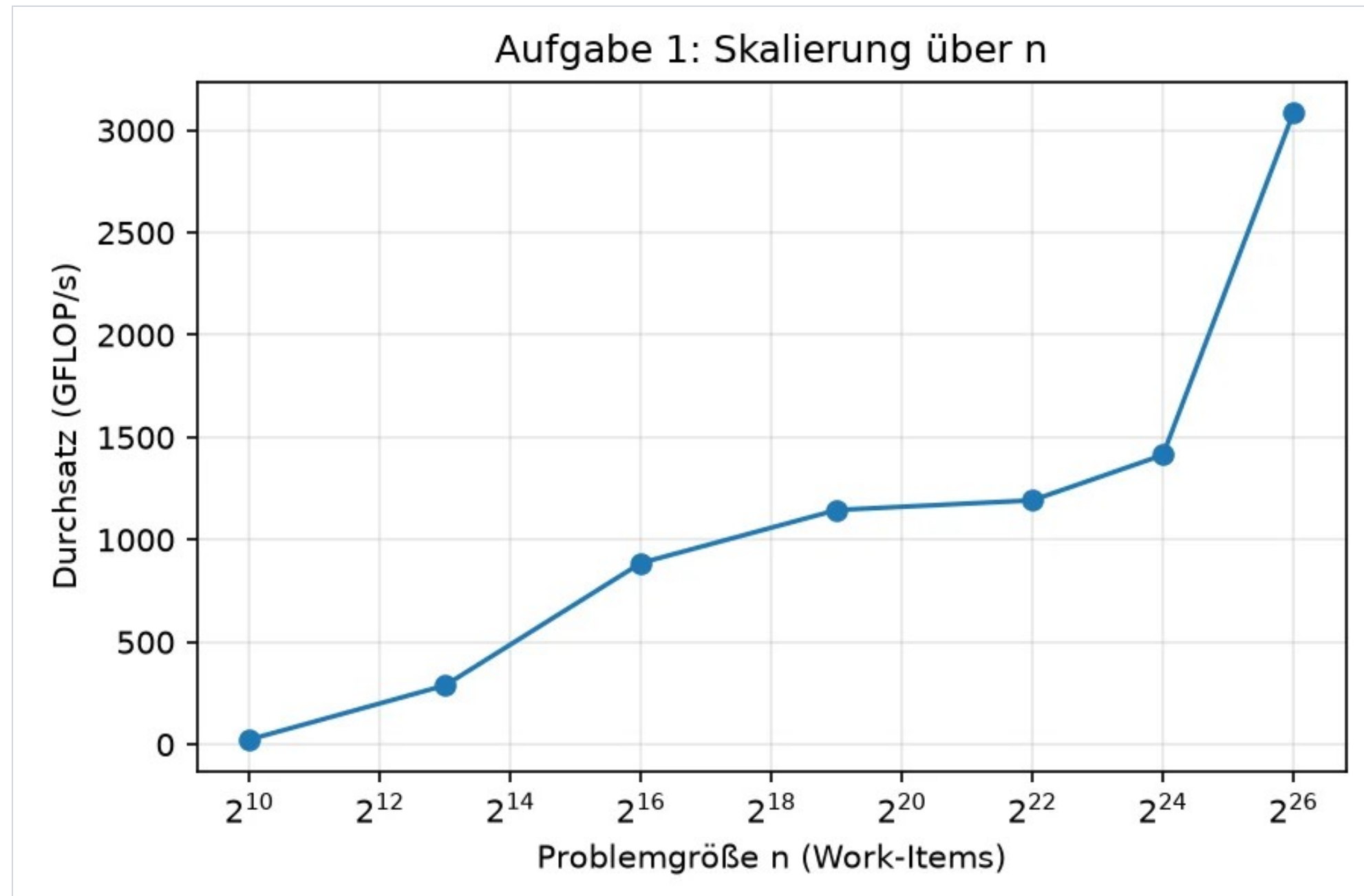
FMA Funktion

```
inline float work(float x) {  
    for (int i = 0; i < KILTERS; ++i) {  
        x = x * COEF + BIAS; // 2 FLOPs  
    }  
    return x;  
}
```

- KILTERS Iterationen, je Schritt eine FMA mit 2 FLOPs
- KILTERS, COEF und BIAS stehen zur Build-Zeit fest, konstante Arbeit je Aufruf

AUFGABE 1

Schritt 1: SIMT trägt erst bei ausreichender Auslastung



$n = 1024$: **23,4 GFLOP/s**

$n \approx 6,7 * 10^7$: **3085,3 GFLOP/s**

- Sättigung, sobald genug Work-Items die 16 Compute Units füllen.

Abbildung 1: Durchsatz über der Problemgröße n, logarithmische n-Achse.

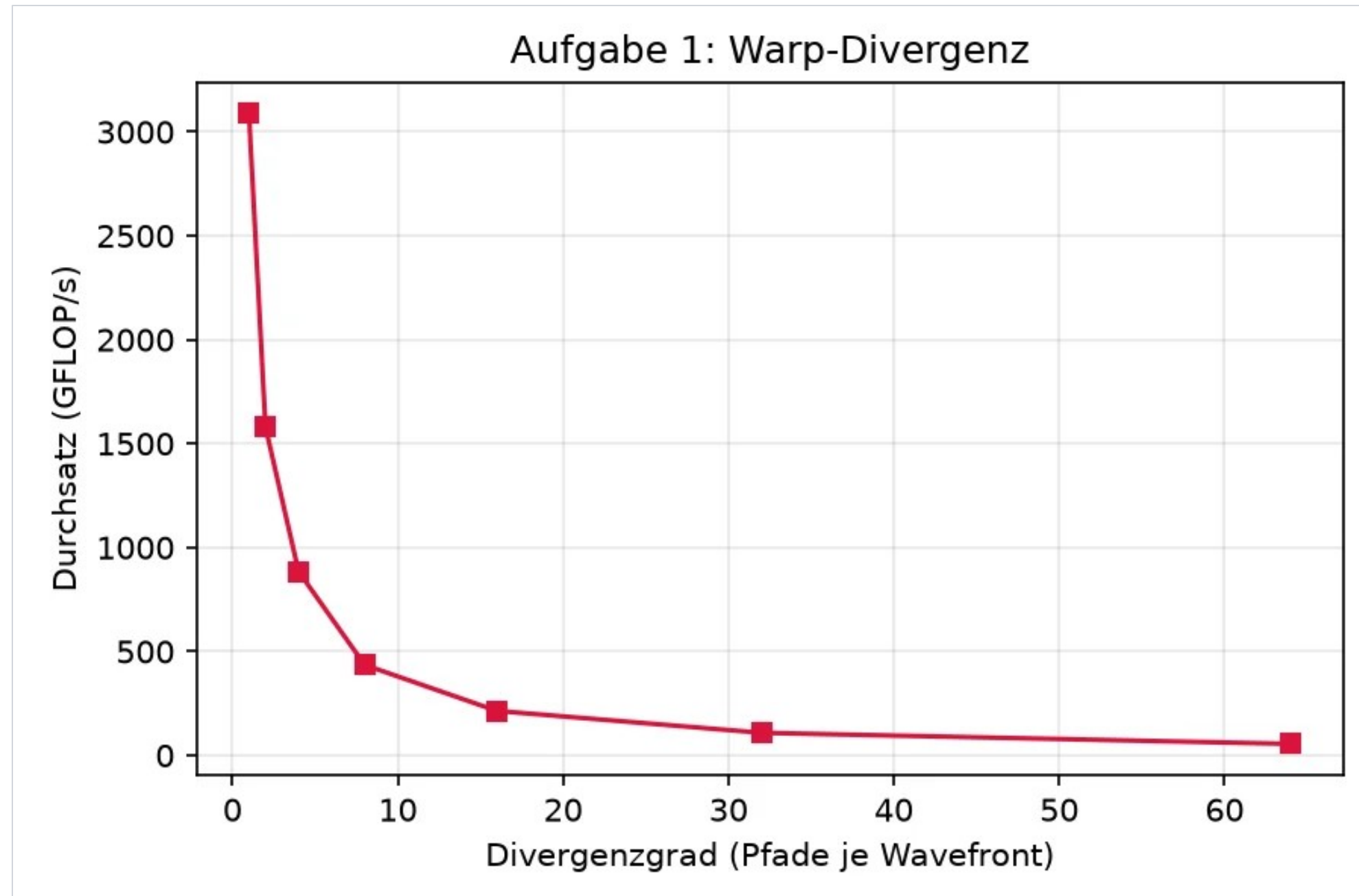
Konstante Arbeit, variable Divergenz

```
int lane = get_local_id(0) % DEGREE;

for (int b = 0; b < DEGREE; ++b) {
    if (lane == b) {
        x = work(x); // KILTERS * 2 FLOPs je Schritt
    }
}
```

- Jedes Work-Item durchläuft die Schleife, trifft aber genau einen Zweig
- FLOP-Zahl je Work-Item bleibt konstant, unabhängig von DEGREE

Schritt 2: Bei gesättigter GPU kostet Divergenz Faktor 56,1



Grad 1: **3088,2 GFLOP/s**

Grad 64: **55,1 GFLOP/s**

- Einbruch um Faktor 56,1 bei konstanter FLOP-Zahl.

Abbildung 2: Durchsatz über dem Divergenzgrad, also der Zahl der Pfade je Warp.

Aufgabe 2: Was kostet unregelmäßiger Speicherzugriff?

- Speicherintensiver Kernel mit sehr geringer arithmetischer Intensität
- Schritt 1: Wie stark unterscheiden sich coalesced, strided und gather?
- Schritt 2: Wie viel Parallelität braucht das beste Muster?
- Messgröße: effektive Bandbreite in GB/s

Ein Kernel, drei Muster, gleiche Byte-Menge

memory.cl

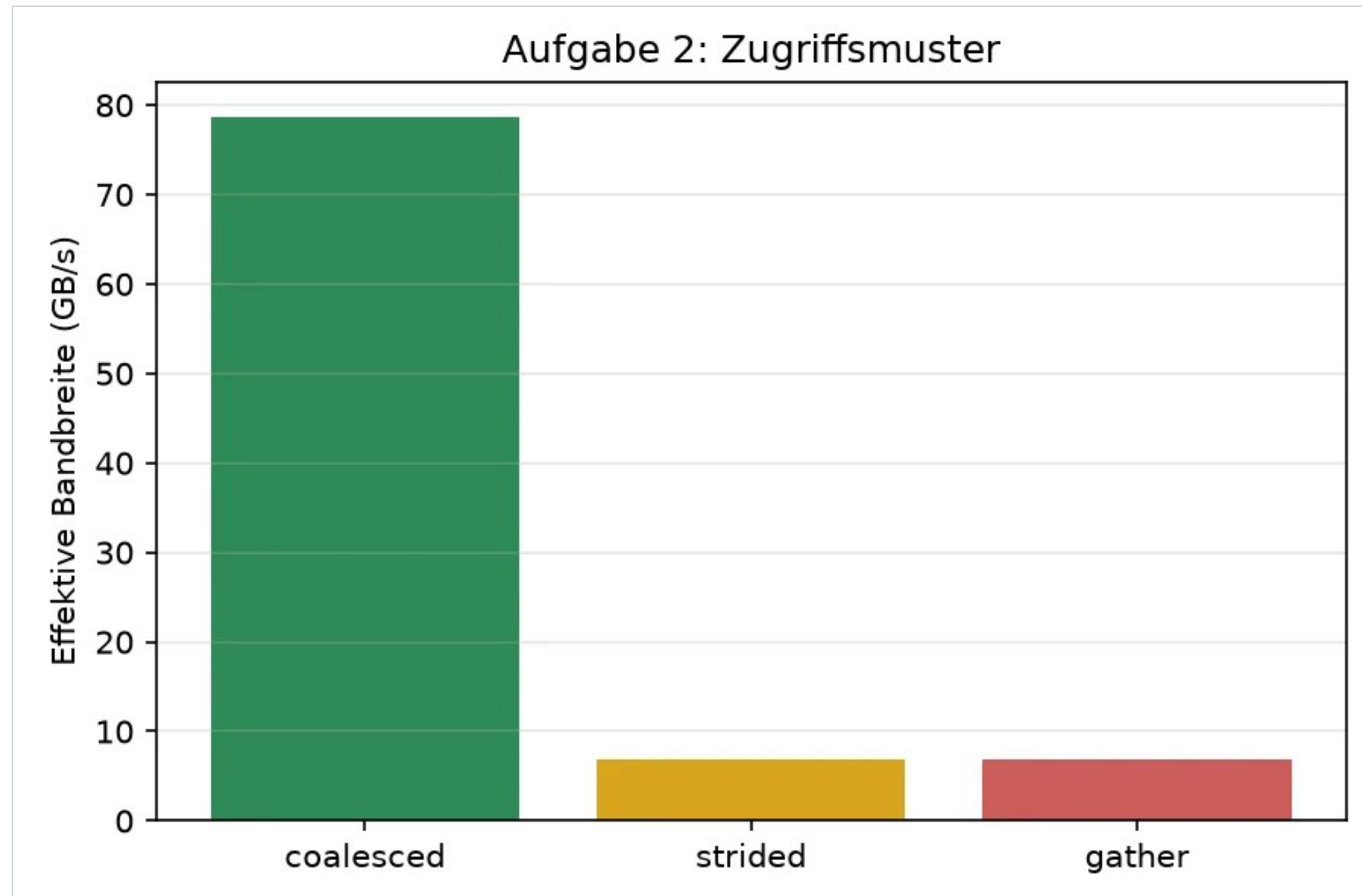
```
b[i] = a[idx[i]] * c;
// Muster steckt allein in idx
```

bench_memory.py

```
coalesced: arange(n)
strided:   (arange(n) * 521) % n
           # 521 prim, teilerfremd zu n
gather:    default_rng(SEED).permutation(n)
```

Muster	Inhalt von idx	Was passiert
coalesced	0 1 2 3 4 5 6 7	Nachbarn lesen Nachbarn
strided	0 3 6 1 4 7 2 5	jeder Zugriff springt 3 weiter
gather	5 2 7 0 3 6 1 4	zufällige Reihenfolge

Schritt 1: Coalescing bestimmt die Bandbreite, nicht die Datenmenge



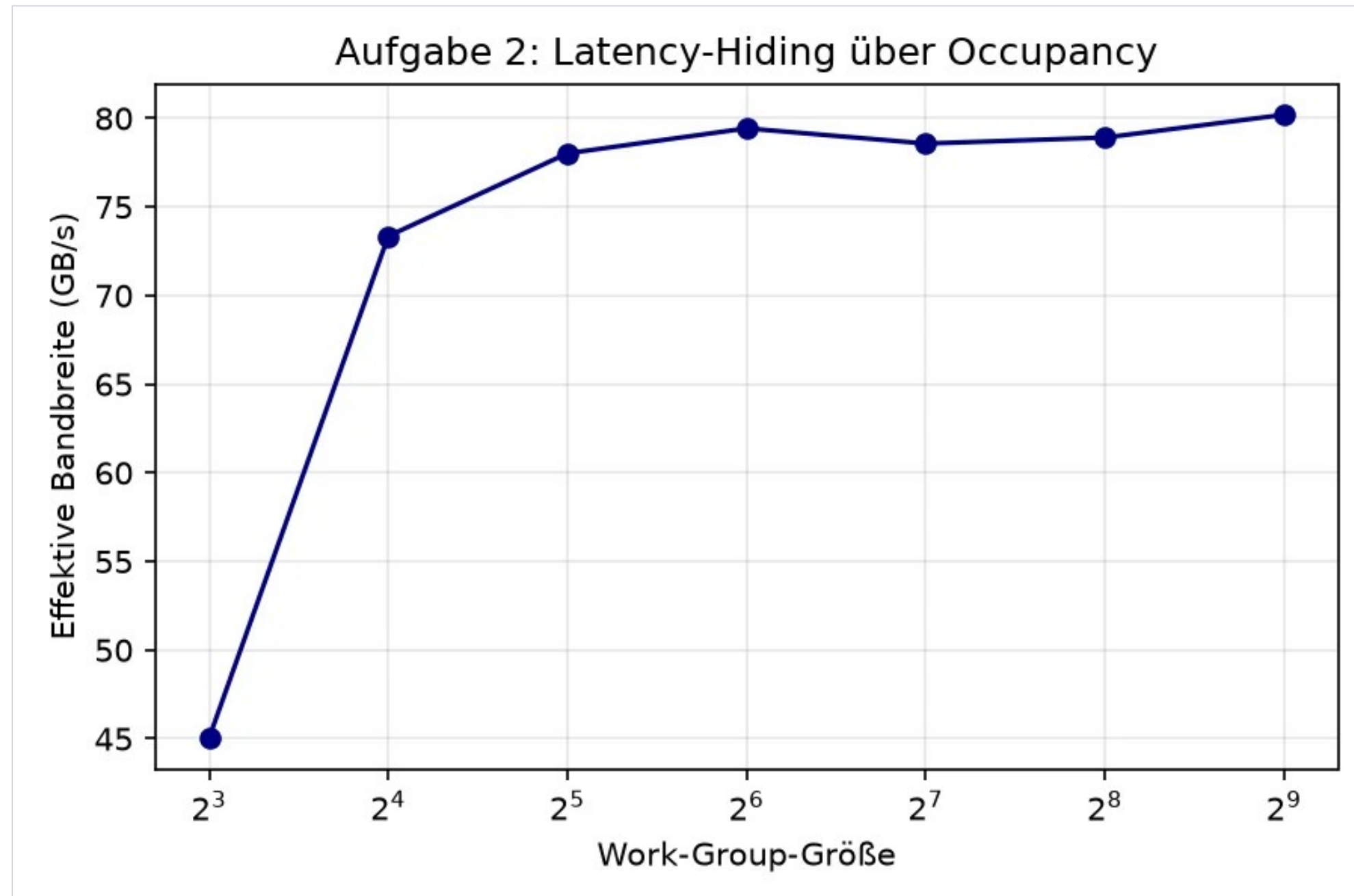
coalesced **78,7 GB/s** , nahe der Kopier-Spitzenbandbreite von 79,4 GB/s

strided **6,9 GB/s** , also Faktor 11,3 langsamer als coalesced

gather **6,9 GB/s** , also Faktor 11,3 langsamer als coalesced

Abbildung 3: Effektive Bandbreite je Zugriffsmuster.

Schritt 2: Bei coalesced-Zugriff sättigt die Bandbreite ab Warp-Breite



Work-Group 8: **45,0 GB/s**

Work-Group 64: **79,4 GB/s**

- Darüber Plateau bis 80,2 GB/s
- Knick liegt bei der Warp-Breite 64

Abbildung 4: Bandbreite über der Work-Group-Größe.

Zwei Regeln folgen aus den Ergebnissen

Messung	GPU	CPU
Spitzen-Durchsatz	3085,3 GFLOP/s	14,5 GFLOP/s
Divergenz-Einbruch Grad 1 zu 64	Faktor 56,1	Faktor 2,6
Bandbreite coalesced	78,7 GB/s	12,9 GB/s
Bandbreite gather	6,9 GB/s	1,5 GB/s

- 1 Verzweigungen innerhalb eines Warps vermeiden
- 2 Datenstrukturen so anordnen, dass benachbarte Threads benachbarte Daten lesen